Problem 1: Euclid's Algorithm

Euclid's algorithm (https://en.wikipedia.org/wiki/Euclidean_algorithm) is a method for finding the greatest common divisor (GCD) of two integers. Recall that the GCD of two integers m and n is the largest number that divides both m and n.

Part 1

The Wikipedia page above includes several pseudocode implementations of Euclid's algorithm. Choose one of these, and use it to implement a function gcd, which takes two integers as its arguments and returns their GCD.

You may assume that both inputs are positive integers (i.e., m, n ∈ Z and m, n > 0), so there is no need to include any error checking in your function.

Note: this is one of the rare occasions where you have my explicit permission to look up your answer. Unless otherwise stated (e.g., as in this problem), looking up solutions on Wikipedia or in any other non-class resource will be considered cheating!

```
#1
def gcd(c, d):
    while d != 0:
        c, d = d, c % d
    return c
print(gcd(18,12))
```

```
6
```

Part 2

What does your function do if one or both of the arguments are negative?

```
#2
print(gcd(48,-18))
print(gcd(-48,18))
print(gcd(-48,-18))

#Python's % operator still works with negative numbers.
#The function will usually return a negative GCD if the first number is negativ
```

```
-6
6
-6
```

Problem 2: Approximating Euler's number e The base of the natural logarithm, e, is typically defined as the infinite sum below: e=∑k=0∞1k!=1+1+12+16+124+...(1)

The term k! denotes the factorial of k, which is k!=k·(k−1)·(k−2)...3·2·1

By convention, 0!=1 For more on Euler's number, see here. In this problem, we will explore different approaches to approximating this number.

Part 1 An early characterization of Euler's number, due to Jacob Bernoulli, was as the limit of

(1+1x)x

as x→∞. Define a function called euler_limit that takes as an argument an integer n, and returns a float that approximates e by taking x=n in Equation (2). You may assume that the input to your function will be a positive integer (of course, your function will still run just fine if n is, say, a positive float, but we will only use integer values in what follows).

```
def euler_limit(n):
    return (1 + 1/n) ** n
print(euler_limit(10))
print(euler_limit(100))
```

```
2.5937424601000023
2.7048138294215285
```

Part 2 What happens when you call euler_limit(n) for really huge values of n (say, 1016 or 1018)? Why does this happen?

Hint: the answer has to do with floating point arithmetic. Think about how 1/n is represented on your computer when n is big.

```
print(euler_limit(10**18))
#When n is extremely large:1/n becomes so small that the computer cannot repres
#so, 1 + 1/n  →  becomes 1.0 and (1.0) ** n = 1.0 so So it incorrectly returns
```

```
1.0
```

Part 3 Define a function called euler_sum that takes a single non-negative integer argument n, and returns an approximation to e based on the first n terms of the sum in Equation (1). Your function should return a float. You may assume that the input will be a non-negative integer, so you do not need to include error checking in your function.

As an example, euler_sum(4) should return the sum of the first four terms in Equation 1, so that euler_sum(4) returns 1+1+1/2+1/6≈2.667 .

Note: the sum in Equation 1 starts counting with k=0 (i.e., it is "0-indexed"), while our function starts counting with n=1 (i.e., it is "1-indexed"). euler_sum(1) should use one term from Equation (1), so that euler_sum(1) returns 1. Similarly, euler_sum(0) should return 0, since by convention an empty sum is equal to zero.

Note: you may use the math.factorial function to compute k! , but I recommend, for the sake of practice, implementing the factorial function yourself, as it is a nice example of a problem that is easily solved with recursion.

```
import math

def euler_sum(n):
    total = 0.0

    for k in range(n):
        total += 1 / math.factorial(k)

    return total

print(euler_sum(4))
print(euler_sum(10))
```

```
2.6666666666666665
2.7182815255731922
```

Part 4 Define a function called euler_approx that takes a single argument, a positive float epsilon, and uses the sum in Equation (1) to obtain an approximation of e that is within epsilon of the true value of e .

Hint: use a while-loop.

Note: you can use the Python math module to get the true value of e (up to floating point accuracy): math.exp(1).

```
import math

def euler_approx(epsilon):
    true_e = math.exp(1)

    total = 0.0
    k = 0

    while abs(total - true_e) > epsilon:
        total += 1 / math.factorial(k)
```

```
            k += 1

        return total

    print(euler_approx(0.0001))
    print(euler_approx(0.5))
```

```
2.7182539682539684
2.5
```

Part 5 For a value of interest x and an approximation x^ , we say the difference x^−x is the error of f(k) .

Define functions called print_euler_sum_error and print_euler_lim_error that take a single positive integer n as input. For k=1,...,n , the function will print the error of the approximation of e given by euler_sum or euler_limit, respectively, evaluated at k . Each error should be printed on its own line (for n lines in total).

```
import math
def factorial(k):
    if k == 0: return 1
    return k * factorial(k - 1)

def euler_limit(n):
    return (1 + 1/n)**n

def euler_sum(n):
    if n == 0: return 0.0
    return sum(1/factorial(k) for k in range(n))

def print_euler_sum_error(n):
    true_e = math.exp(1)
    for k in range(1, n + 1):
        approximation = euler_sum(k)
        error = approximation - true_e
        print(error)

def print_euler_lim_error(n):
    true_e = math.exp(1)
    for k in range(1, n + 1):
        approximation = euler_limit(k)
        error = approximation - true_e
        print(error)

print("Errors for the Sum Method:")
print_euler_sum_error(5)

print("\nErrors for the Limit Method:")
print_euler_lim_error(5)
```

```
Errors for the Sum Method:
-1.718281828459045
-0.7182818284590451
-0.2182818284590451
-0.05161516179237857
-0.00994849512571161

Errors for the Limit Method:
-0.7182818284590451
-0.4682818284590451
-0.3479114580886753
-0.2768755784590451
-0.22996182845904567
```

Part 6 Which approximation is better?

```
The Euler Sum Method is better than the Euler Limit Method
```

Problem 3: Testing properties of integers In this problem, you'll get a bit more practice working with conditionals, and a first exposure to the kind of thinking that is required in a typical "coding interview" question. A positive integer n is a power of 2 if n=2p for some integer p≥0 .

Part 1 Write a function is_power_of_2 that takes a positive integer as its only argument and returns a Boolean indicating whether or not the input is a power of 2.

You may assume that the input is a positive integer. You may not use the built-in math.sqrt function in your solution. You should need only the division and modulus (%) operations.

Hint: the simplest solution to this problem makes use of recursion, though recursion is not necessary.

```
def is_power_of_2(n):

    while n > 1:

        if n % 2 != 0:
            return False

        n = n // 2

    return True

print(is_power_of_2(1))
print(is_power_of_2(4))
```

```
True
True
```

Part 2 Generalize your previous solution to a function is_power that takes two positive integers as its arguments, b and n, in that order, and returns a Boolean. is_power(b,n) should return True if n is a power of b and False otherwise.

```
def is_power(b, n):

    if n == 1:
        return True

    if n % b != 0:
        return False

    return is_power(b, n // b)
print(is_power(5, 125))
```

```
True
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.